

Redes neuronales desde el perceptrón hasta el umbral de Transformers

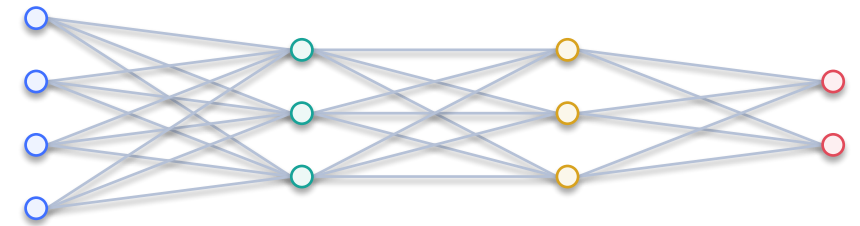
INTELIGENCIA ARTIFICIAL - CATEDRA.

Profesores: Mg. Diaz, Santiago Raúl · Lic. Cardoso, Pedro · A.S.

Lopez, Joaquin, BioIng. Hadad A

$$\hat{y} = f_{\theta}(x), \quad \theta \leftarrow \theta - \eta \nabla_{\theta} L$$

**Idea central: aprender en IA = ajustar
parámetros para minimizar una función
de error.**



Perceptrón

MLP

RNN

Transformers

Hoja de ruta ampliada de la clase

El foco es matemático: linealidad, no linealidad, minimización del error, backpropagation y ejemplos numéricos.

Preguntas guía

¿Por qué el perceptrón simple sólo resuelve separación lineal?

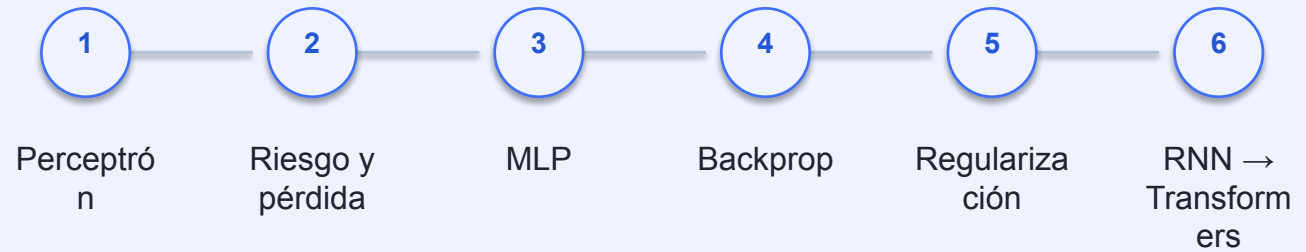
¿Qué cambia al introducir capas ocultas y activaciones?

¿Cómo se define la función de error en clasificación y regresión?

¿Cómo calcula backpropagation los gradientes?

¿Qué problema temporal intentan resolver las RNN y cómo conectan con Transformers?

Recorrido



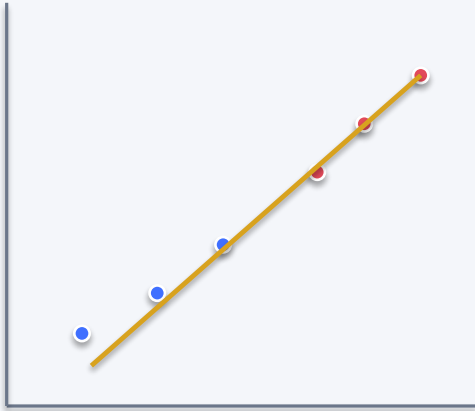
Durante la clase aparecen actividades breves de comprensión en el medio y ejemplos numéricos completos.

Intuición inicial: linealidad vs. aproximación funcional

El perceptrón simple define una frontera plana; una MLP busca una función compuesta más expresiva.

Perceptrón simple = frontera lineal

$$\hat{y} = \text{sign}(w^\top x + b)$$

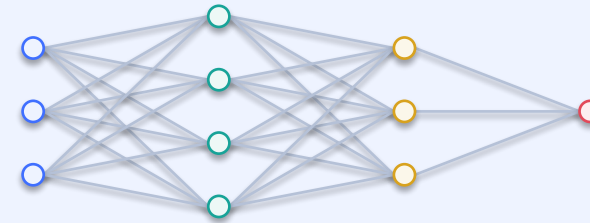


Si las clases se separan con una recta (o hiperplano), un solo perceptrón alcanza.

La decisión depende sólo del signo de una combinación lineal.

Red multicapa = función compuesta

$$f_{\theta}(x) = W^{(L)}\sigma(W^{(L-1)}\sigma(\dots\sigma(W^{(1)}x + b^{(1)})\dots) + b^{(L-1)}) + b^{(L)}$$



Con no linealidades, la red puede curvar, combinar y recombinar fronteras simples.

Mensaje clave: apilamos transformaciones lineales y no lineales para aproximar funciones complejas.

Aprender = minimizar una función de error

Teóricamente minimizamos riesgo esperado; en práctica minimizamos riesgo empírico sobre una muestra.

Riesgo esperado e integral

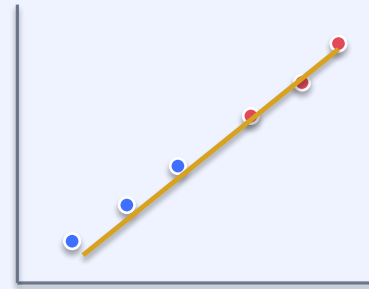
$$R(\theta) = E_{(x,y) \sim p(x,y)}[\ell(f_\theta(x), y)] = \int \ell(f_\theta(x), y) p(x, y) dx dy$$

Como la distribución real $p(x,y)$ es desconocida, la reemplazamos por una muestra finita observada.

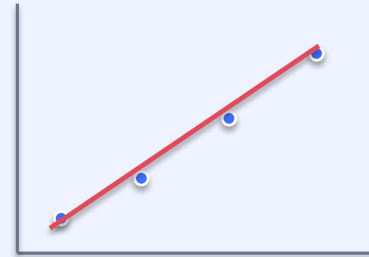
$$\hat{R}_n(\theta) = \frac{1}{n} \sum_{i=1}^n \ell(f_\theta(x_i), y_i), \quad \theta^{(t+1)} = \theta^{(t)} - \eta \nabla_\theta \hat{R}_n(\theta^{(t)})$$

Toda la materia cabe aquí: modelo + pérdida + gradiente.

Clasificación y regresión



Clasificación: salida discreta, clase o probabilidad.



Regresión: salida continua, por ejemplo precio, demanda o temperatura.

Perceptrón simple: modelo, geometría y decisión

La condición de decisión es el signo de una suma ponderada.

Modelo

$$a = w^T x + b = \sum_{j=1}^d w_j x_j + b$$

$$\hat{y} = \text{sign}(a)$$

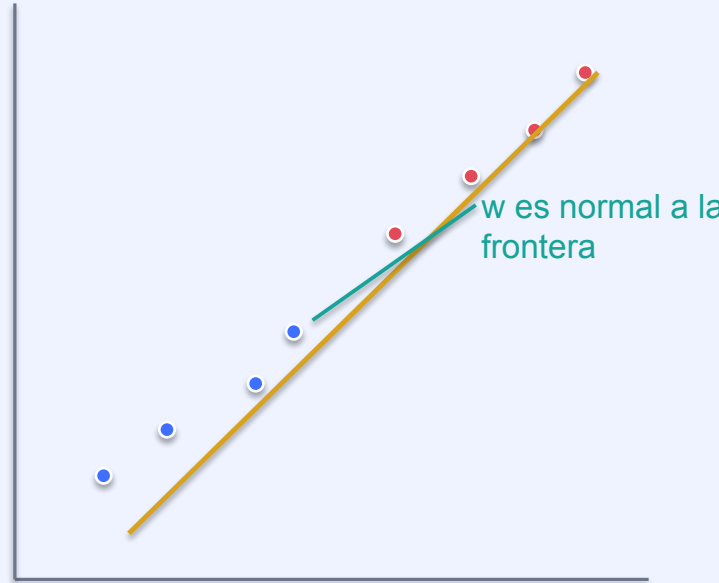
Si $a > 0$ cae de un lado de la frontera; si $a < 0$ cae del otro.

$$w^T x + b = 0$$

$$\text{dist}(x, H) = \frac{|w^T x + b|}{\|w\|_2}$$

La distancia al hiperplano mide la “confianza” geométrica.

Lectura geométrica en 2D



Todo punto del mismo lado de la recta recibe la misma decisión. Por eso el perceptrón simple no captura fronteras curvas o combinadas.

Teorema de convergencia del perceptrón

Cuando los datos son linealmente separables con margen positivo, el algoritmo converge en cantidad finita de errores.

Enunciado

Si existe w^* tal que $y_i(w^{*T}x_i) \geq \gamma > 0$ para todo i y $\|x_i\| \leq R$, el número de errores del perceptrón satisface:

$$M \leq \left(\frac{R}{\gamma}\right)^2$$

Consecuencia: si hay separabilidad lineal, el algoritmo termina.

Idea de la prueba

Cada actualización aumenta la proyección de w sobre el separador ideal w^* .

Al mismo tiempo, la norma de w no crece arbitrariamente rápido.

Comparando ambas cotas se obtiene el límite superior para M .

$$w^{(t+1)} = w^{(t)} + y_i x_i$$

Lectura conceptual

El margen γ mide cuánta “holgura” tienen los datos respecto de la frontera. Cuanto más grande es γ , más fácil es aprender.

Si el conjunto NO es separable, el teorema deja de aplicar: el algoritmo puede oscilar.

Esto motiva pérdidas suaves y redes multicapa.

Algoritmo del perceptrón y criterio de error

La actualización sólo se dispara cuando un ejemplo es mal clasificado.

Regla clásica

$$\text{si } y_i(w^\top x_i + b) \leq 0, \quad w \leftarrow w + \eta y_i x_i, \quad b \leftarrow b + \eta y_i$$

Si el ejemplo es correcto, no hay cambio.

Si es incorrecto, la corrección mueve la frontera hacia la clase adecuada.

η es la tasa de aprendizaje.

Puede interpretarse como descenso por subgradiente de un criterio simple.

Criterio del perceptrón

$$L_P(w, b) = - \sum_{i \in \mathcal{M}} y_i (w^\top x_i + b)$$

donde $\mathcal{M}$ es el conjunto de ejemplos mal clasificados.

$$\partial_w L_P = - \sum_{i \in \mathcal{M}} y_i x_i, \quad \partial_b L_P = - \sum_{i \in \mathcal{M}} y_i$$

Observación importante: la función escalón no es derivable. Por eso las redes modernas usan activaciones derivables o subderivables y pérdidas suaves.

Ejemplo numérico 1: perceptrón para la compuerta AND

La compuerta AND es linealmente separable, por lo que un único perceptrón alcanza.

Datos

$(0,0) \rightarrow 0$

$(0,1) \rightarrow 0$

$(1,0) \rightarrow 0$

$(1,1) \rightarrow 1$

$$\hat{y} = \text{step}(w_1x_1 + w_2x_2 + b)$$

Inicialización: $w=(0,0)$, $b=0$, $\eta=1$.

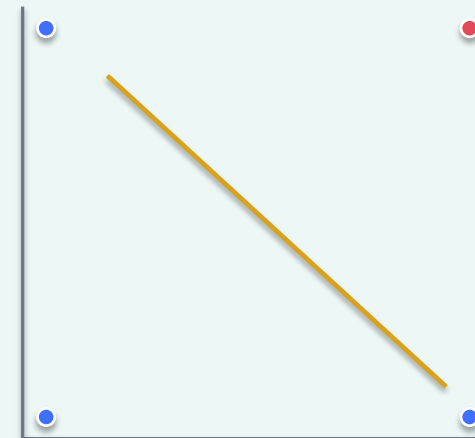
Una solución final válida es
 $w=(1,1)$, $b=-1.5$.

Secuencia breve de actualizaciones

- 1) $x=(0,0)$, $y=0 \rightarrow$ sin cambio
- 2) $x=(1,1)$, $y=1 \rightarrow w=(1,1)$, $b=1$
- 3) $x=(1,0)$, $y=0 \rightarrow$ error $\rightarrow w=(0,1)$, $b=0$
- 4) $x=(0,1)$, $y=0 \rightarrow$ error $\rightarrow w=(0,0)$, $b=-1$
- 5) nuevas pasadas $\rightarrow$ converge a una frontera correcta

Lo importante no es la secuencia exacta sino que la frontera se corrige con cada error.

Plano x_1-x_2

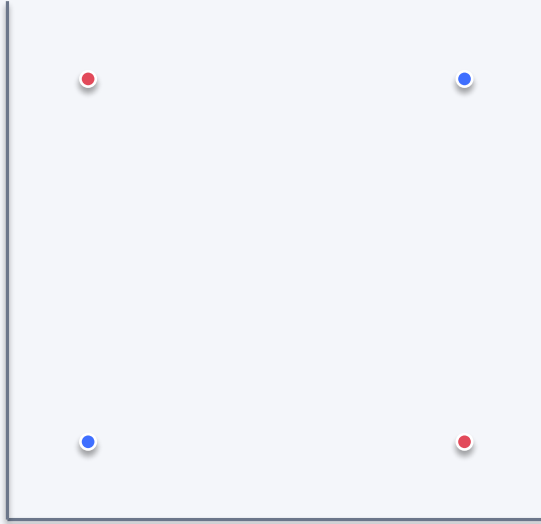


La recta $x_1+x_2-1.5=0$ deja sólo al punto $(1,1)$ del lado positivo.

Actividad breve 1: ¿se puede separar linealmente?

Detectar cuándo un problema necesita más que un solo hiperplano.

XOR



Preguntas para 5 minutos:

- ¿Puede una sola recta separar estas clases?
- Si no puede, ¿qué le falta al perceptrón simple?
- ¿Qué estrategia te sugiere una capa oculta?

Pista: la clase roja es una unión de dos regiones.

Discusión esperada

XOR no es linealmente separable.

Un único perceptrón sólo produce una frontera plana.

Dos o más neuronas ocultas pueden crear regiones intermedias y luego combinarlas.

Ésta es la primera razón profunda para pasar del perceptrón simple a una MLP: necesitamos componer fronteras simples para obtener una función más rica.

Por qué una capa sin no linealidad no alcanza

La no linealidad no es un detalle: evita que la red colapse en una sola transformación afín.

Proposición

$$h^{(1)} = W^{(1)}x + b^{(1)}, \quad \hat{y} = W^{(2)}h^{(1)} + b^{(2)}$$

$$\hat{y} = \tilde{W}x + \tilde{b}, \quad \tilde{W} = W^{(2)}W^{(1)}, \quad \tilde{b} = W^{(2)}b^{(1)} + b^{(2)}$$

Es decir: la composición de funciones afines sigue siendo afín.

Prueba breve: sustituir $h^{(1)}$ en la segunda ecuación y reagrupar términos.

Consecuencia práctica



Si σ es la identidad, toda la red equivale a un solo hiperplano. No se resuelve XOR ni ninguna estructura genuinamente no lineal.

Por eso aparecen sigmoide, tanh y ReLU.

Funciones de activación y sus derivadas

Sin no linealidad no hay expresividad; sin derivada útil no hay aprendizaje por gradientes.

Escalón

$$H(z) = 1 \text{ si } z \geq 0, 0 \text{ si } z < 0$$

No derivable en 0; útil para decisión.

Sigmoide

$$\sigma(z) = \frac{1}{1+e^{-z}}, \sigma'(z) = \sigma(z)(1 - \sigma(z))$$

Produce probabilidades; puede saturarse.

tanh

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}, \tanh'(z) = 1 - \tanh^2(z)$$

Centrada en 0; también satura.

ReLU

$$\text{ReLU}(z) = \max(0, z), \text{ReLU}'(z) = 1 \text{ si } z > 0, 0 \text{ si } z < 0$$

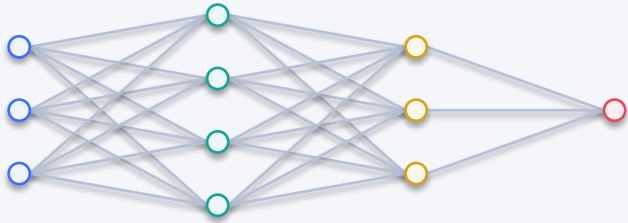
Simple y efectiva en redes profundas.

Idea clave: sin no linealidad, no hay salto de expresividad; sin derivada o subderivada, no hay aprendizaje eficiente.

Red multicapa (MLP): propagación hacia adelante

Cada capa calcula una combinación lineal seguida de una activación.

Arquitectura



Entrada → capas ocultas → salida

Cada capa aprende una representación intermedia.

Ecuaciones

$$h^{(0)} = x$$

$$a^{(\ell)} = W^{(\ell)} h^{(\ell-1)} + b^{(\ell)}$$

$$h^{(\ell)} = \sigma(a^{(\ell)}), \quad \ell = 1, \dots, L-1$$

$$\hat{y} = g(a^{(L)}), \quad a^{(L)} = W^{(L)} h^{(L-1)} + b^{(L)}$$

Regresión: g suele ser identidad.

Clasificación binaria: g suele ser sigmoide.

Multiclase: g suele ser softmax.

El forward produce predicciones; el backward explica cómo corregir parámetros.

Teorema de aproximación universal

Una MLP con una capa oculta suficientemente ancha puede aproximar funciones continuas sobre compactos.

Enunciado útil para clase

Sea K un compacto de $\mathbb{R}^d$ y $f:K \rightarrow \mathbb{R}$ continua. Para ciertas activaciones no polinómicas y cualquier $\varepsilon > 0$, existe una red de una capa oculta tal que:

$$g(x) = \sum_{j=1}^m \alpha_j \sigma(w_j^T x + b_j)$$

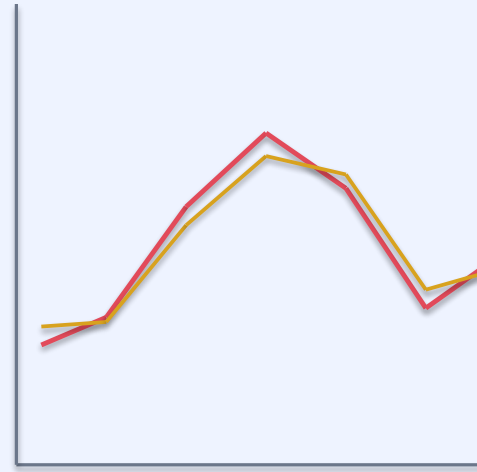
$$\max_{x \in K} |f(x) - g(x)| < \varepsilon$$

No dice que encontrar esos parámetros sea fácil.

No dice cuántas neuronas mínimas hacen falta.

Sí dice que la familia de funciones es muy rica.

Intuición



Rojo: función objetivo.
Dorado: aproximación.

La red suma “bloques” no lineales que, combinados, pueden copiar casi cualquier curva continua sobre un dominio acotado.

Pérdidas y capas de salida para clasificación y regresión

Elegir la pérdida correcta equivale a decidir qué error queremos penalizar.

Regresión

$$\ell_{MSE}(\hat{y}, y) = \frac{1}{2}(\hat{y} - y)^2$$

$$\frac{\partial \ell}{\partial \hat{y}} = \hat{y} - y$$

La MSE castiga errores grandes al cuadrado; es natural cuando la salida es continua.

Ejemplos: demanda, precio, temperatura.

Clasificación binaria

$$\hat{p} = \sigma(a), \quad \ell_{BCE}(\hat{p}, y) = -[y \log \hat{p} + (1 - y) \log(1 - \hat{p})]$$

$$\frac{\partial \ell}{\partial a} = \hat{p} - y$$

La BCE penaliza fuerte cuando el modelo asigna baja probabilidad a la clase correcta.

Multiclase + regularización

$$\hat{p}_k = \frac{e^{a_k}}{\sum_j e^{a_j}}, \quad \ell = - \sum_{k=1}^K y_k \log \hat{p}_k$$

$$L_\lambda(\theta) = \hat{R}_n(\theta) + \lambda \|\theta\|_2^2$$

El término $\lambda \|\theta\|^2$ controla complejidad y reduce sobreajuste.

Optimización por gradiente: batch, mini-batch y SGD

El entrenamiento moderno aproxima el riesgo empírico con lotes pequeños.

Ecuaciones

$$g_t = \frac{1}{|B_t|} \sum_{(x_i, y_i) \in B_t} \nabla_{\theta} \ell(f_{\theta_t}(x_i), y_i)$$

$$\theta_{t+1} = \theta_t - \eta_t g_t$$

Batch: usa todos los datos.

SGD: usa un solo ejemplo.

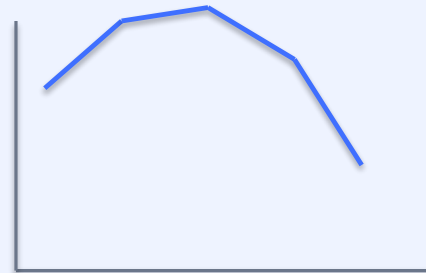
Mini-batch: equilibrio estándar en deep learning.

Observaciones

En MLP profundas el problema ya no es convexo.

Si η es demasiado grande, la optimización puede divergir.

Si η es demasiado pequeña, el aprendizaje se vuelve muy lento.



**Pensarlo como “bajar”
una superficie de error.**

Backpropagation: idea general y regla de la cadena

Backprop reutiliza gradientes locales para evitar derivar toda la red desde cero.

Grafo computacional mínimo



En backward calculamos cómo cambia ℓ si cambia cada variable intermedia.

Regla de la cadena matricial

$$\delta^{(L)} = \nabla_{a^{(L)}} \ell = \nabla_{\hat{y}} \ell \odot g'(a^{(L)})$$

$$\delta^{(\ell)} = (W^{(\ell+1)})^\top \delta^{(\ell+1)} \odot \sigma'(a^{(\ell)})$$

$$\frac{\partial \ell}{\partial W^{(\ell)}} = \delta^{(\ell)} (h^{(\ell-1)})^\top, \quad \frac{\partial \ell}{\partial b^{(\ell)}} = \delta^{(\ell)}$$

Interpretación: el error se “propaga hacia atrás” multiplicando derivadas locales.

Derivación matemática de backprop en una red de una capa oculta

Caso típico de clase: salida sigmoide con pérdida de entropía cruzada binaria.

Forward

$$a^{(1)} = W^{(1)}x + b^{(1)}, \quad h = \sigma(a^{(1)})$$

$$a^{(2)} = W^{(2)}h + b^{(2)}, \quad \hat{y} = \sigma(a^{(2)})$$

$$\ell = -[y \log \hat{y} + (1 - y) \log(1 - \hat{y})]$$

Esta será la red para el ejemplo numérico siguiente.

Backward

$$\frac{\partial \ell}{\partial a^{(2)}} = \left(-\frac{y}{\hat{y}} + \frac{1-y}{1-\hat{y}}\right) \hat{y}(1-\hat{y}) = \hat{y} - y$$

$$\delta^{(2)} = \hat{y} - y$$

$$\frac{\partial \ell}{\partial W^{(2)}} = \delta^{(2)} h^\top$$

$$\delta^{(1)} = (W^{(2)})^\top \delta^{(2)} \odot \sigma'(a^{(1)})$$

$$\frac{\partial \ell}{\partial W^{(1)}} = \delta^{(1)} x^\top$$

Ejemplo numérico 2: un paso completo de backprop

Red mínima: una entrada, una neurona oculta ReLU y una salida sigmoide.

Datos y parámetros

$$x = 1, y = 1$$

$$w = 0.8, b_h = -0.3$$

$$v = 1.2, b_o = 0.1$$

$$\eta = 0.1$$

Activación oculta: ReLU.
Pérdida: BCE.

Paso 1: forward

$$a_h = 0.8 \cdot 1 - 0.3 = 0.5$$

$$h = \text{ReLU}(0.5) = 0.5$$

$$a_o = 1.2 \cdot 0.5 + 0.1 = 0.7$$

$$\hat{y} = \sigma(0.7) \approx 0.668$$

$$\ell = -\log(0.668) \approx 0.403$$

Como $y=1$ y la probabilidad predicha fue baja, el gradiente empuja a subir v y w .

Paso 2: backward y update

$$\delta_o = \hat{y} - y = -0.332$$

$$\frac{\partial \ell}{\partial v} = \delta_o h \approx -0.166$$

$$\delta_h = v \delta_o \text{ReLU}'(0.5) = 1.2(-0.332) = -0.398$$

$$\frac{\partial \ell}{\partial w} = \delta_h x \approx -0.398$$

$$v' = 1.2166$$

$$w' = 0.8398$$

Actividad breve 2: interpretar el signo del gradiente

La intuición sobre signos y magnitudes evita memorizar fórmulas sin entender.

Consigna

Supongamos una clasificación binaria con sigmoide. Un peso produce $\partial \ell / \partial w = -0.15$.

- 1) Si $\eta=0.1$, ¿el update aumenta o disminuye w ?
- 2) ¿Qué indica el signo sobre la dirección de corrección?
- 3) ¿Qué pasa si el módulo del gradiente es grande?

$$w_{\text{nuevo}} = w - \eta \frac{\partial \ell}{\partial w}$$

Discusión esperada

Como el gradiente es negativo, el update suma: $w_{\text{nuevo}} > w$.

El signo indica hacia qué lado mover el parámetro para bajar la pérdida.

Un gradiente grande produce un cambio grande salvo que η sea pequeña o apliquemos clipping/regularización.

Meta cognitiva: poder “leer” una derivada como una señal de corrección.

Un ejemplo cercano a la realidad: riesgo de deserción estudiantil

Un caso natural para Sistemas de Información: combinar múltiples señales institucionales.

Variables de entrada

$x = [\text{asistencia, promedio, accesos LMS, tardanzas}]^T$

La etiqueta puede ser binaria: deserta / no deserta. O continua: nota final esperada.

$$\hat{p}(\text{desercion} \mid x) = \sigma(w^T x + b)$$

Perceptrón como baseline

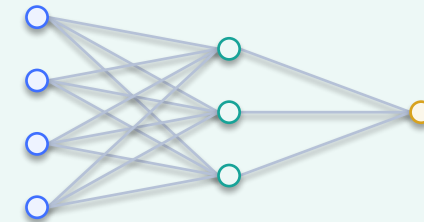
Rápido y explicable.

Útil si el efecto de cada variable es aproximadamente lineal.

Falla si hay interacciones complejas entre variables.

$$p = \sigma(0.7x_1 - 1.1x_2 - 0.9x_3 + 0.5x_4 + b)$$

MLP para capturar interacciones



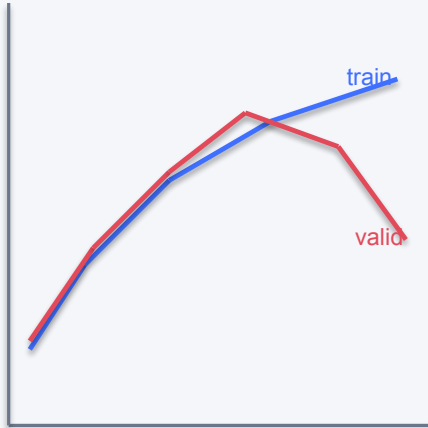
La capa oculta puede aprender combinaciones como “riesgo alto” cuando varias señales se degradan simultáneamente.

En producción también importan calidad de datos, sesgos, balanceo y métricas (recall, AUC, MAE, etc.).

Generalización: sobreajuste, regularización y estabilidad

No alcanza con ajustar bien la muestra: queremos buen desempeño en datos no vistos.

Sobreajuste



La curva de entrenamiento o baja, pero la validación vuelve a subir: el modelo memoriza peculiaridades del conjunto de entrenamiento.

Regularización

$$L_{\lambda}(\theta) = \hat{R}_n(\theta) + \lambda \|\theta\|_2^2$$

L2: penaliza pesos grandes.

Early stopping: detener cuando validación deja de mejorar.

Dropout: apaga unidades al azar durante entrenamiento.

Gradientes inestables

$$\prod_{\ell} W^{(\ell)} \sigma'(a^{(\ell)})$$

En redes profundas, productos de derivadas muy pequeñas o muy grandes hacen que el gradiente se desvanezca o explote.

Prácticas útiles: inicialización cuidadosa, normalización, learning rate razonable y gradient clipping.

Redes recurrentes (RNN) y puente a Transformers

Las RNN introducen memoria temporal; Transformers eliminan la recurrencia estricta con atención.

RNN

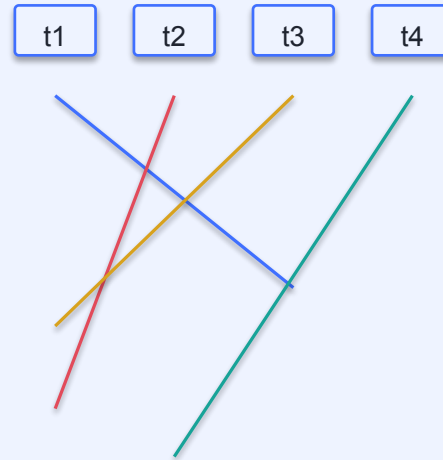
$$h_t = \phi(W_{xh}x_t + W_{hh}h_{t-1} + b_h)$$

$$\hat{y}_t = g(W_{hy}h_t + b_y)$$

El estado oculto resume pasado y procesa secuencias como texto, series temporales o audio.

$$\frac{\partial L}{\partial h_t} = \sum_{k \geq t} \frac{\partial L_k}{\partial h_k} \prod_{j=t+1}^k \frac{\partial h_j}{\partial h_{j-1}}$$

Dependencias directas



En atención, cada token puede conectarse con los demás sin pasar por una cadena recurrente larga.

Transformers: teaser

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Mayor paralelismo computacional.

Mejor captura de dependencias largas.

Base de los LLMs y modelos multimodales.

Próxima clase: embeddings, self-attention, multi-head attention y arquitecturas encoder/decoder.

Cierre: ideas fuerza y bibliografía recomendada

Resumen conceptual para llevarse de la clase.

Ideas fuerza

El perceptrón simple resuelve problemas linealmente separables porque su frontera es un hiperplano.

Una red multicapa busca una función compuesta; sin activaciones no lineales colapsa a una sola transformación afín.

Aprender equivale a minimizar una pérdida; backpropagation aplica la regla de la cadena para calcular gradientes eficientemente.

Clasificación y regresión cambian salida y pérdida, no la lógica central de optimización.

RNN agregan memoria temporal; Transformers reemplazan la recurrencia por atención.

Bibliografía mínima

- Goodfellow, Bengio y Courville — Deep Learning
- Bishop — Pattern Recognition and Machine Learning
- Hastie, Tibshirani y Friedman — The Elements of Statistical Learning
- Nielsen — Neural Networks and Deep Learning
- Vaswani et al. — Attention Is All You Need

Sugerencia didáctica: complementar con práctica en Python implementando perceptrón y una MLP pequeña para visualizar el descenso del error.